PRACTICAL ASSIGNMENT 2: CLUSTERING RANDOM DATA

Christoph Van Jaarsveld

STUDENT NUMBER: 50143956

Table of Contents

Project Code

```
# Christoph Van Jaarsveld 50143956

# Import needed libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Define dataset parameters
# Number of observations in each class
N1 = 300
N2 = 300
# Distance between class centres
d = 2.0

# Standard deviation
sigma1 = 1.5
sigma2 = 1.5

# Generate class center locations
# Class center 1 coordinates
x1_center = d / 2 * np.array([-1, 1])

# Class center 2 coordinates
x2_center = d / 2 * np.array([1, 1])

# Create feature Matrix
# Combine class center locations
X_T = np.vstack((x1_center, x2_center))

# Add noise to class centres to create data points
noise1 = np.random.normal(scale=sigma1, size=(N1, 2))
noise2 = np.random.normal(scale=sigma2, size=(N2, 2))

# Add noise to class centres to create data points
X1 = X_T[0, :] + noise1
X2 = X_T[1, :] + noise2

# Combine data points from both classes
X = np.concatenate ((X1, X2), axis=0)

# Generate Class Labels
# Create class labels (0 for class 1, 1 for class 2)
y = np.array([0] * N1 + [1] * N2)

# Visualizse and cluster data
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
```

```
plt.title("Generated Dataset")
plt.show()

# Define the number of clusters
n_clusters = 2

# Create a KMeans object with the specified number of clusters
kmeans = KMeans(n_clusters=n_clusters)

# Fit the KMeans model to the data
kmeans.fit(X)

# Get the cluster labels assigned by KMeans
kmeans_labels = kmeans.labels_

# Align KMeans labels with true class labels
aligned_labels = np.zeros_like(kmeans_labels)
for cluster in range (n_clusters):
  mask = (kmeans_labels == cluster)
  aligned_labels[mask] = np.argmax(np.bincount(y[mask]))

# Calculate accuracy
accuracy = np.sum(aligned_labels == y) / len(y)
print("Accuracy: ", accuracy)

# Visualise the KMeans clustering results
plt.scatter(X[:, 0], X[:, 1], c=kmeans_labels, cmap='viridis')
plt.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            c='red', s=100, marker='X', label='Centroids') # Mark
centroids
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("KMeans Clustering Result")
plt.legend()
plt.show()
```

Project Overview

In this practical we were provided with code sections that generated a synthetic 2D dataset of two Gaussian distributed classes. Including the task of experimenting and observing the results of our own substitute values such as the following.

- Number of observations in each class
- Distance between class centers
- Standard deviations

The code applies the KMeans clustering algorithm to the generated dataset to group the two clusters. The clustered results are then visualised with scatter plots and clustering accuracy is calculated by aligning predicted cluster labels with the true class labels

Current Code and Results

Dataset parameters used

```
# Number of clusters
n_clusters = 2


# Number of observations in each class
N1 = 300
N2 = 300

# Distance between class centres
d = 2.0


# Standard deviation
sigma1 = 1.5
sigma2 = 1.5


# Class centers
Class 1: [-1, 1]
Class 2: [1, 1]
```

Generated Dataset Scatter Plot

With these parameter values used there are two clusters that are horizontally separated. Class 1 being centred at (-1, 1) and Class 2 being centred at (1,1).

Standard deviations sigma1 and sigma2 are relatively moderate, showing noticeable spread in both clusters. Due to the small distance between the two clusters (d = 2.0), there is some overlap between the clusters.

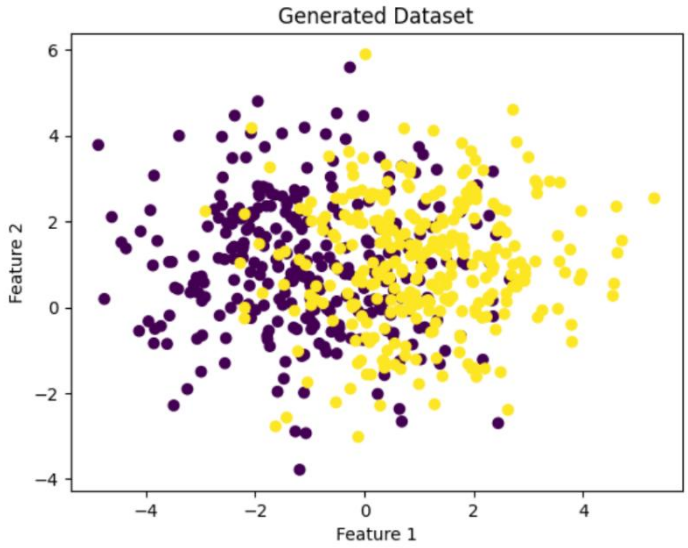

Figure 1: Generated Dataset

KMeans Clustering Result

After applying the KMeans algorithm to the two clusters most of the points are correctly grouped with a accuracy score of (0.82). The accuracy score is high but not perfect due to some minor overlap.

It is clear that the clustering worked well. The data formed exactly two noticeable spherical clusters with a reasonable separation.

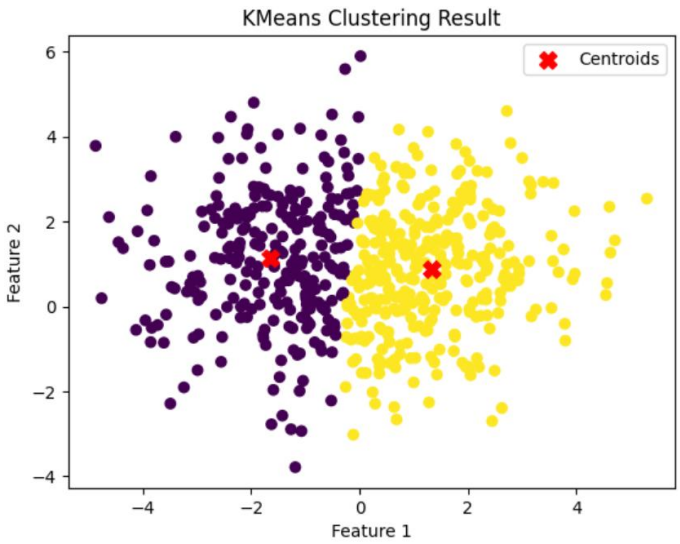

Figure 2: KMeans Clustering Result

Parameter Experiments and Observations

Experiment 1: Increasing Distance Between Class Centers

Parameters Changed:

- d = 6.0
- All other parameters unchanged

What Happens to the Graph:

The two clusters become clearly separated, moving much further along the x-axis with a gap between them. Overlap between the classes is eliminated resulting in a significant increase in accuracy of 100%.

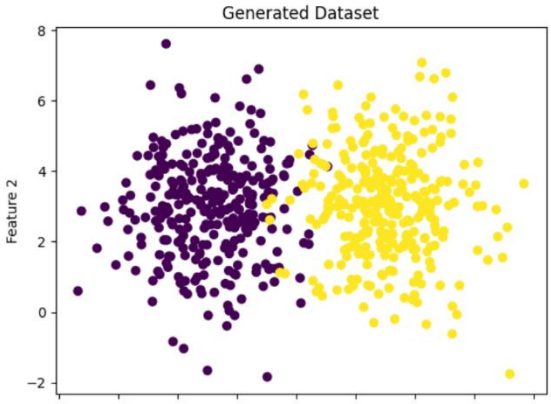

Figure 3: Experiment 1Generated Dataset

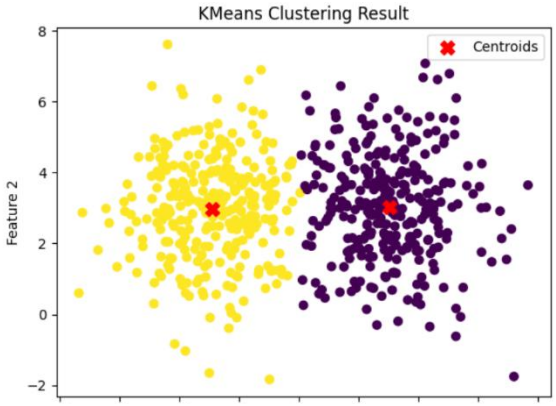

Figure 4: Experiment 1 KMeans Clustering Result

Experiment 2: Increasing standard Deviation

Parameters Changed:

- Sigma1 = 2.5
- Sigma2 = 2.5
- All other parameters unchanged

What Happens to the Graph:

Data points scatter further from their centres, resulting in the two clusters being much wider and more spread out. The two classes become heavily overlapped and the boundary between the clusters becomes unclear. With a noticeable decrease in accuracy score to (0.63).

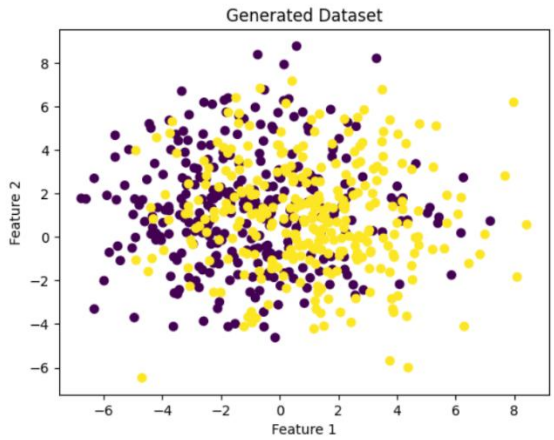

Figure 5: Experiment 2 Generated Dataset

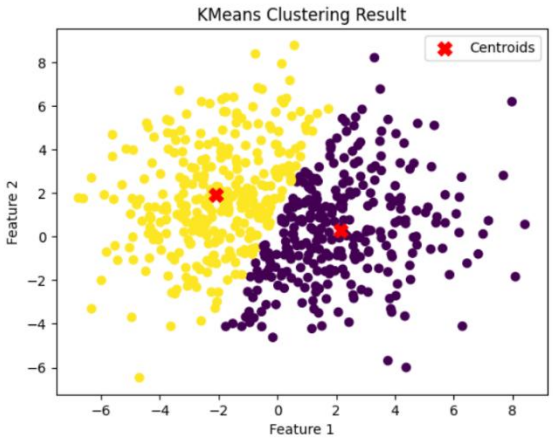

Figure 6: Experiment 2 KMeans Clustering Result

Experiment 3: Unequal Number of Observations

Parameters Changed:

- N1 = 100
- N2 = 600
- All other parameters unchanged

What Happens to the Graph:

One cluster becomes more populated, visually dominating the graph. The centroid positioning becomes biased towards the larger group. KMeans centroids shifts slightly towards the larger cluster due to its greater influence.

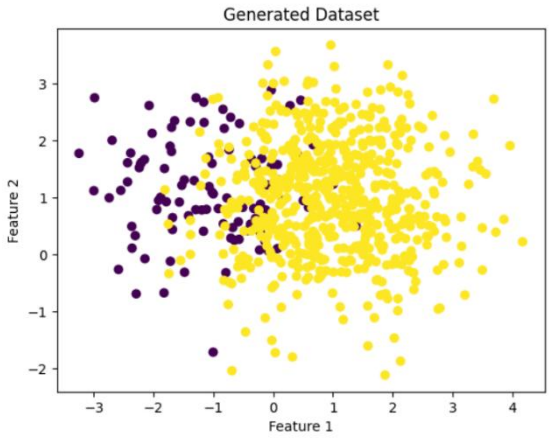

Figure 7: Experiment 3 Generated Dataset

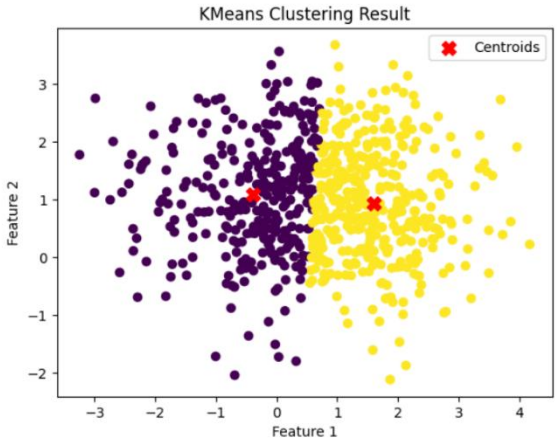

Figure 8: Experiment 3 KMeans Clustering Result

Conclusion

This experiment shows that clustering performance strongly depends on data distribution parameters. Understanding how these parameters affect cluster structure is essential for interpreting real world clustering results.

Key findings:

- Increasing distance between clusters improves clustering accuracy
- Increasing standard deviation increases overlap and reduces accuracy
- Class imbalance influences centroid positioning but does not necessarily destroy clustering if separation exists